Build Task — Put a UI on a Real Pipeline

> Back to the Build Task · Hint 1 · Hint 2 · Solution

Goal: take one real pipeline you already built — InvoiceIQ (Project 14), MeetingMind (Project 15), JobMatch (Project 16), or any agent — and give it **two** web layers: a **Streamlit** app (the demo) and a **Flask or FastAPI** app (the real service). The pipeline is imported unchanged; only the web files are new.

Hint 1

The rule that makes this easy: **the pipeline never changes.** Keep it in its own module with one entry function — e.g. InvoiceIQ's `process_invoice(model, path)` or a thin wrapper `analyze(text) -> dict`. Both web files just `import` it and call it.

```
pipeline.py        -> analyze(text) -> {"summary": ..., "problems": [...]}
app_streamlit.py   -> import analyze; build the UI around it
app_web.py         -> import analyze; expose it as a route
```

If your pipeline needs a model/config, build it once at startup (module level or a cached resource) so every request reuses it — don't re-create it per call.

Hint 2

Validate at the web edge, and give slow calls feedback:

- **Empty input** → show "please enter…" (Streamlit) or return `400` / `422` (Flask/FastAPI), never a crash.
- **A real pipeline is slow** → Streamlit's `with st.spinner("Working…"):` around the call; an API can return a result directly if it's quick, or a job id if it's long.
- **The same input to both UIs must give the same result** — because both call the identical pipeline. That's the test that proves the web layer added no logic.

Solution

A worked version wrapping a small `analyze` pipeline (swap in a real project's function — the web code is the same shape).

```
# pipeline.py — stands in for a real project's entry function
def analyze(text: str) -> dict:
    lowered = text.lower()
    problems = []
    if "urgent" in lowered:
        problems.append("flagged as urgent")
    if len(text.strip()) < 3:
        problems.append("input too short")
    mood = "neutral"
    for word in ["love", "great", "good"]:
        if word in lowered:
            mood = "positive"
    for word in ["bad", "hate", "terrible"]:
        if word in lowered:
            mood = "negative"
    return {"mood": mood, "problems": problems}
```

The Streamlit demo:

```
# app_streamlit.py   —   streamlit run app_streamlit.py
import streamlit as st

from pipeline import analyze

st.title("Analyzer")
text = st.text_area("Paste text:")
if st.button("Analyze"):
    if not text.strip():
        st.warning("Please enter some text.")
    else:
        with st.spinner("Working…"):
            result = analyze(text)
        st.write("Mood:", result["mood"])
        if result["problems"]:
            st.write("Problems:")
            for problem in result["problems"]:
                st.write("-", problem)
        else:
            st.success("No problems.")
```

The FastAPI service (the shippable version):

```
# app_web.py   —   uvicorn app_web:app --reload
from fastapi import FastAPI
from pydantic import BaseModel

from pipeline import analyze

app = FastAPI()

class AnalyzeRequest(BaseModel):
    text: str

@app.post("/analyze")
def analyze_route(body: AnalyzeRequest):
    if not body.text.strip():
        return {"error": "empty text"}
    return analyze(body.text)
```

The test that proves both agree (and the edge cases):

```
# test_build.py   —   python -m test_build
from fastapi.testclient import TestClient

from app_web import app
from pipeline import analyze

def test_same_result_and_edges() -> None:
    client = TestClient(app)
    text = "I love this but it is urgent"
    direct = analyze(text)
    api = client.post("/analyze", json={"text": text}).json()
    empty = client.post("/analyze", json={"text": ""}).json()
    result = "OK"
    if api != direct:
        result = "WRONG"
    if "error" not in empty:
        result = "WRONG"
    print(f"case 1 (both agree; empty handled) -> {result}")

if __name__ == "__main__":
    test_same_result_and_edges()
```

Expected: the API result equals the direct `analyze(...)` result (the web layer added nothing); empty input returns an `error`, not a crash. The Streamlit app shows the same mood and problems with a spinner during the call.

Which would you ship? For a quick internal demo, the Streamlit app. For a real service a frontend or another team calls, the FastAPI version — typed, validated, documented. Both wrap the *same* pipeline, which is the whole point: the AI is done, and the web layer is a thin, swappable wrapper you choose to fit the job.